

ESPECIALIZACIÓN EN **INTELIGENCIA DE DATOS** para la Gestión Estratégica

TIPOS DE DATOS, ARCHIVOS Y FORMATOS

Clase 1 · Taller T8001 — Pre-procesado de datos

Docente: Dr. Lautaro Di Bartolo

Asignatura: T8001 — Pre-procesado de datos

Cohorte o comisión: Cohorte 2026

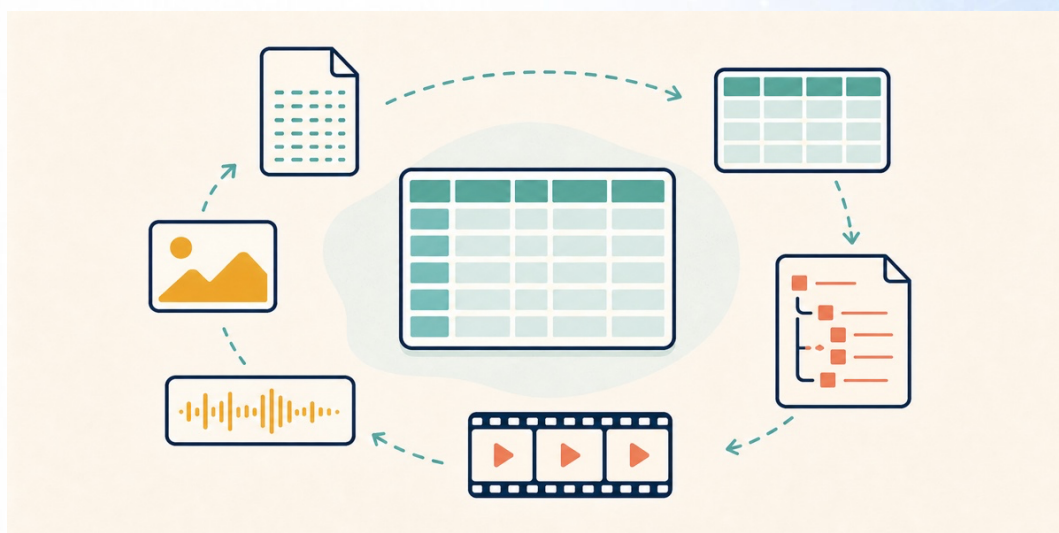


Figura 1. Distintos contenidos y formatos, atravesados por el mismo trabajo de preparación.

1. INTRODUCCIÓN

Entre el 60% y el 80% del tiempo de un proyecto de datos se va en conseguir los datos, entenderlos y prepararlos. El número sale de encuestas y varía, pero el orden de magnitud no se discute: el análisis, que es la parte visible, ocupa la minoría del trabajo.

Este apunte cubre la primera mitad de ese trabajo: de dónde sale un archivo, cómo está armado por dentro, qué se pierde al convertirlo y por qué a veces los acentos aparecen rotos. Los ejemplos son tablas de cuatro filas, inventadas para que se puedan leer de un vistazo.

El eje del taller es el criterio sobre los datos y no la programación. Casi ninguna decisión de pre-procesado tiene una única respuesta correcta, así que lo que importa es poder justificar la que se tomó.

2. HERRAMIENTAS

2.1 Google Colab

Colab es un servicio gratuito de Google que ejecuta código Python en una máquina de Google, desde el navegador. No hay nada que instalar y no importa qué computadora tengas. Hace falta una cuenta de Google.

Lo que se abre en Colab es un **notebook**: un documento con dos tipos de celda. Las de texto explican, y son la mayor parte. Las de código ejecutan Python y muestran el resultado justo abajo. Para ejecutar una celda, hacé clic adentro y apretá Shift + Enter.

Cuatro cosas que conviene saber antes de abrir el primero:

- **Guardá una copia primero. Archivo → Guardar una copia en Drive.** Si no lo hacés, los cambios se pierden cuando cerrás la pestaña.
- **Se ejecuta en orden, de arriba hacia abajo.** Cada celda usa lo que dejaron las anteriores. Si una celda da un resultado raro, casi siempre es porque se ejecutaron en desorden.
- **Cuando algo se desordena, empezá de nuevo. Entorno de ejecución → Reiniciar y ejecutar todo.** Tarda un minuto y resuelve la mayoría de los problemas.
- **Los archivos están en el panel de la izquierda.** Buscá el ícono de carpeta. Ahí ves lo que el notebook descargó y lo que generó.

Hay un tutorial de tres minutos, con doblaje al español: [Introducción a Google Colab](#).

2.2 Programación en Python

Este taller no es un curso de programación. El código de las demostraciones ya está escrito, y lo que se pide es leerlo y entender qué devuelve.

Una aclaración de vocabulario, porque la vas a necesitar. Python por sí solo no sabe abrir un archivo de datos ni calcular el promedio de una columna. Para eso están las **librerías**: paquetes de código que ya escribió otra gente y que uno trae a su programa. En todo el taller usamos una sola para los datos, y se llama **pandas**. Es la herramienta más usada para trabajar con tablas en Python, y cuando el notebook diga `import pandas as pd`, eso es lo que está haciendo.

Para que quede claro qué quiere decir leer el código. Una línea típica del notebook es esta:

```
df = pd.read_csv("crudo/hogares.csv")
```

Tiene tres partes. `pd.read_csv` es una herramienta de pandas que abre un archivo CSV. Entre paréntesis va cuál. Y el resultado queda guardado con el nombre `df`, que es el que usan las celdas de abajo. `df` viene de *dataframe*, que es como pandas llama a una tabla. Un bloque rojo debajo de una celda es un error y no un desastre: se lee el último renglón, que dice qué pasó.

Si nunca escribiste ni leíste una línea, la ruta corta son tres capítulos del [curso de Python de MoureDev](#), gratuito y en español: [Introducción](#) (4 minutos), [Hola Mundo](#) (23 minutos) y [Variables](#) (45 minutos). El tercero es el que más rinde: qué es una variable y qué tipos de datos hay.

3. QUÉ ES EL PRE-PROCESADO

3.1 Por qué hace falta

Los datos casi nunca se generan para el análisis que uno quiere hacer. Se generan para facturar, para cumplir un trámite, para operar un sistema. Vos llegás después, con otra pregunta, y el dato no está en la forma que necesitás: le faltan valores, tiene la misma categoría escrita de tres maneras, las fechas vienen como texto y hay filas repetidas.

Un caso. Un municipio quiere saber cuántos hogares distintos visitó un programa social en marzo. El único dato que existe es la planilla con la que administración le liquida el viático a cada encuestador. Tiene una fila por visita cobrada, y no por hogar: el hogar visitado dos veces aparece dos veces, y el hogar del encuestador que todavía no presentó el reintegro no aparece. La localidad la escribe cada uno a mano, así que Concepción, Concepcion y Concep. son la misma. Columna de hogar no hay: hay una dirección en texto libre.

Ninguna de esas cosas es un defecto de la planilla. Para liquidar viáticos anda perfecto. Para contar hogares hay que sacar los repetidos, unificar las localidades, decidir qué pasa con las visitas no cobradas y convertir la dirección en un identificador.

El pre-procesado es el trabajo de llevar ese dato desde la forma en que llegó hasta una forma en la que se puede analizar, y de dejar registrado qué se hizo en el camino.

3.2 Qué es un dato, y de qué tipos

Un dato es un registro de algo que pasó, guardado de una forma que una máquina puede leer. La temperatura de ayer, el monto de una factura, la foto de una radiografía, la grabación de una llamada. Todo eso son datos, y todos terminan siendo números en un disco.

Se los suele agrupar en tres familias, según cuánta estructura traen:

- **Estructurados.** Vienen en tablas, con filas y columnas definidas. Una planilla de sueldos, la base de clientes de un sistema. Son los más cómodos y los que más se usan en este taller.
- **Semiestructurados.** Tienen una organización interna, pero no una tabla rectangular. Un archivo JSON que devuelve una página web, un documento XML. Se pueden convertir a tabla, y esa conversión es parte del pre-procesado.
- **No estructurados.** No tienen ninguna organización que un programa pueda aprovechar de entrada: imágenes, sonido, video, texto libre. Son la mayoría de los datos que existen, y para analizarlos hay que extraerles primero alguna variable medible.

El semiestructurado es el que más cuesta ver. Este es el hogar de Ramallo tal como lo devuelve una página web:

```
{"id": 1, "localidad": "Ramallo", "personas": 3, "servicios": ["agua", "luz", "gas"]}
```


Hay nombres de campo y hay valores, así que estructura tiene. Tabla no: servicios guarda tres valores en un solo campo, y el hogar que sigue puede traer dos, ninguno, o un campo que este no tiene. Nada obliga a que todos los registros tengan las mismas columnas. Convertirlo a tabla es decidir qué hacer con eso, y la sección 3.5 tiene la respuesta.

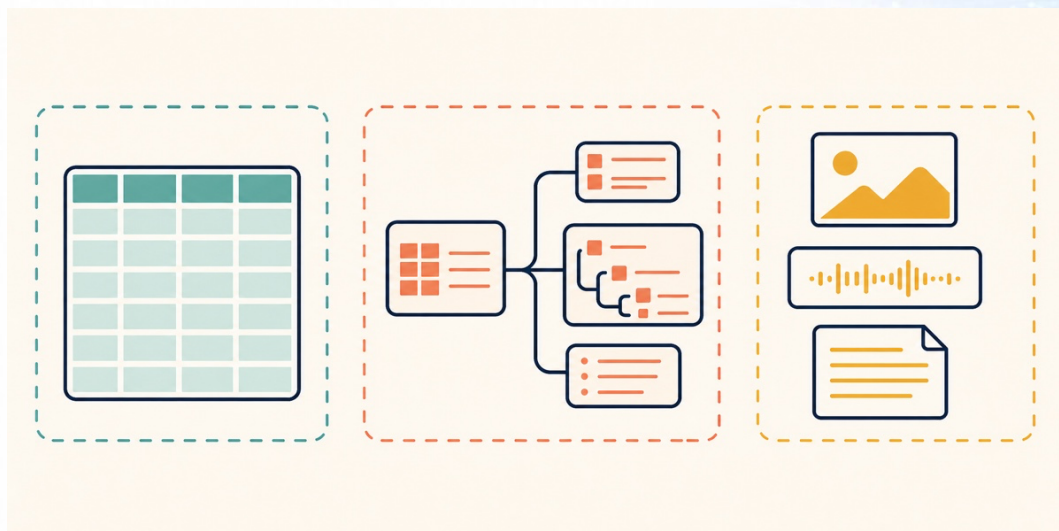


Figura 2. De una estructura fija a contenidos que primero hay que interpretar.

La sección 7 vuelve sobre los no estructurados.

3.3 Qué es un archivo de datos

Un archivo es una tira de bytes guardada en un disco. Un **byte** es un número de 0 a 255, y nada más. Un archivo son millones de esos números, uno atrás del otro.

La extensión del nombre (.csv, .xlsx, .jpg) es una convención y no una garantía. Le dice al sistema operativo con qué programa abrirlo, pero nada impide que adentro haya otra cosa. Un archivo llamado datos.csv puede contener una planilla de Excel, y va a fallar cuando lo intentes leer como CSV.

Lo que sí define qué hay adentro es el **formato**: el acuerdo sobre cómo se ordenan esos bytes. La sección 5 compara cinco formatos distintos para la misma tabla.

3.4 El CSV por dentro

El formato con el que vas a trabajar casi todo el taller es el **CSV**, de *comma-separated values*, valores separados por comas. Es el más simple de todos y por eso el más universal.

Un CSV entero se ve así:

```
id,localidad,personas,ambientes,fecha_visita,ingreso
1,Ramallo,3,2,2025-03-14,185000
2,Córdoba,5,3,2025-03-15,240500
3,Concepción,2,1,2025-03-15,
4,Ramallo,4,2,2025-03-16,198000
```

Eso es todo. Es **texto plano**, del mismo tipo que un .txt: lo abre cualquier editor y se puede leer con los ojos. La primera línea tiene los nombres de las columnas, cada línea que sigue es una fila, y las comas separan un valor del siguiente. Una coma al final de la línea, como en la fila con id 3, significa que el último valor está vacío.

El CSV tiene un estándar escrito, el [RFC 4180](#), y de ahí lo que más importa es una regla: un campo que contiene una coma, una comilla o un salto de línea va entre comillas dobles. Sin ellas, Rosario, Santa Fe se leería como dos columnas.

Su simpleza es también su límite. Como es texto sin ninguna marca, el archivo no dice en qué encoding está escrito ni qué carácter usa como separador de campos, que no siempre es la coma. De esos dos silencios salen los problemas de la sección 6.

3.5 La regla cardinal de los datos tabulares

Tres condiciones definen qué es una tabla usable. Las dos primeras vienen de [*Tidy Data*](#) (Wickham, 2014); la tercera, del programa de Data Carpentry.

1. Una variable por columna.
2. Una observación por fila.
3. Un valor por celda.

Antes de seguir, tres grupos de sinónimos que vas a ver mezclados en cualquier texto del área: variable = columna = campo; observación = fila = caso = registro; conjunto de datos = dataset.

Suena obvio hasta que se ve roto. Esta tabla incumple la tercera condición:

id	localidad	servicios
1	Ramallo	agua;luz;gas
2	Córdoba	agua;luz

Para contar cuántos hogares tienen gas hay que partir el texto de cada celda. La versión que respeta la regla usa una fila por combinación:

id	servicio
1	agua
1	luz
1	gas
2	agua
2	luz

Esta otra incumple la primera:

localidad	2023	2024	2025
Ramallo	320	410	380
Córdoba	512	498	540

El año es un dato, no un nombre de columna. Cada vez que llegue un año nuevo hay que agregar una columna y reescribir todo lo que lea esa tabla. La versión ordenada tiene tres columnas fijas:

localidad	año	valor
Ramallo	2023	320
Ramallo	2024	410
Ramallo	2025	380
Córdoba	2023	512
Córdoba	2024	498
Córdoba	2025	540

Pedile a la tabla ancha el promedio por año. Hay que nombrar 2023, 2024 y 2025 una por una, y reescribir esa línea cada vez que llegue un año nuevo. Sobre la tabla ordenada es una instrucción sola: agrupar por año y promediar valor, y sale igual con tres años que con veinte. Con un gráfico de la serie pasa lo mismo: la forma ordenada tiene una columna para el eje horizontal y la ancha no tiene ninguna.

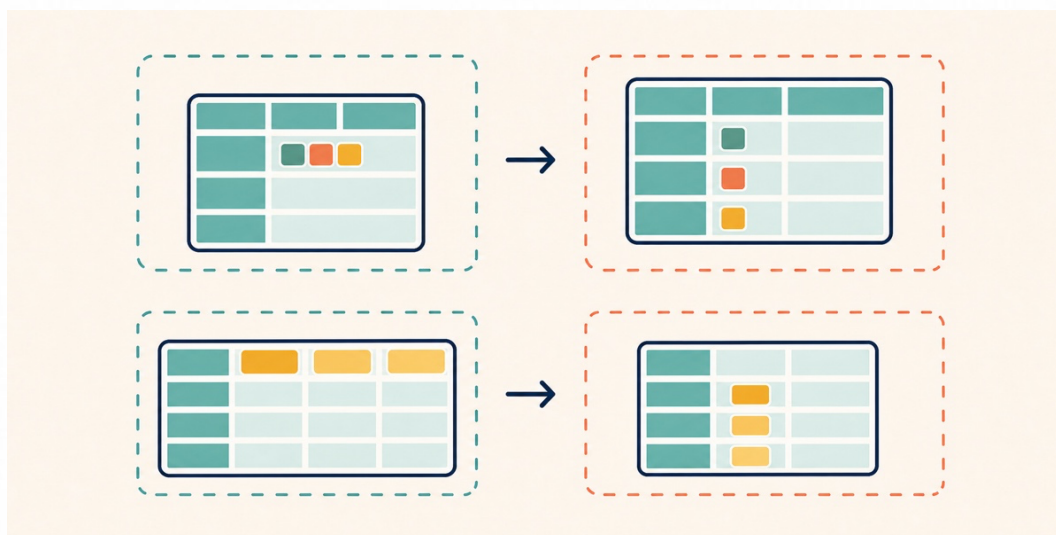


Figura 3. Cada condición rota se arregla moviendo el dato que estaba en el lugar equivocado.

Una aclaración, para no volverse dogmático. La tabla ancha con un año por columna se lee mejor para una persona, y la regla no la prohíbe: dice que esa es una forma de presentación y no la forma de guardar el dato.

Una tabla que cumple las tres condiciones se puede filtrar, agrupar y graficar con una línea. Una que no, obliga a escribir código de rescate antes de empezar.

3.6 La planilla real no es una tabla

Una planilla hecha para que la lea una persona casi nunca arranca con los nombres de las columnas. Arriba hay un título, un logo y dos filas en blanco. Abajo hay una fila de totales y una nota al pie. Y en el medio hay celdas combinadas, que agrupan a la vista lo que la máquina necesita ver repetido en cada fila.



Figura 4. Una planilla puede parecer una tabla sin comportarse como una.

Al leer ese archivo hay que decirle al programa en qué hoja está la tabla, en qué fila empieza el encabezado y hasta dónde llegan los datos, y sacar la fila de totales, porque mezclada con las observaciones arruina cualquier promedio. Sacarla tarde tampoco es gratis: la herramienta ya eligió el tipo de cada columna con esa fila adentro, y no lo corrige sola. La [guía de publicación de datos abiertos de la Nación](#) le pide lo mismo al que publica: sin totales, sin subtotales y sin celdas combinadas.

3.7 Cuándo un dato es de buena calidad

La calidad de un conjunto de datos se puede evaluar. Estas son las siete dimensiones habituales, y vuelven a aparecer en el trabajo final. Se ven mejor sobre una tabla que las incumple todas:

id	localidad	personas	fecha_visita	ingreso
1	Ramallo	3	2025-03-14	185000
2	Córdoba	5	2025-03-15	240500
2	Córdoba	5	2025-03-15	240500
3	Concepción	2	15/03/2025	
4	ramallo	4	2025-03-16	198000

- **Complejidad.** Falta el ingreso del hogar 3.
- **Consistencia.** Ramallo y ramallo son la misma localidad.
- **Validez.** La fecha del hogar 3 no sigue el formato de las otras.
- **Unicidad.** El hogar 2 está dos veces.
- **Exactitud.** El ingreso del hogar 1 dice 185000 y el recibo dice 285000.
- **Oportunidad.** La tabla llega en agosto y el subsidio se define en abril.
- **Trazabilidad.** Nadie sabe quién la armó ni cuándo.

Las cuatro primeras se ven en el archivo. Las tres últimas no: hay que ir a la fuente o preguntar. Exactitud y validez se confunden. Una edad de 45 en un campo de edad es válida. Si esa persona tiene 32, es inexacta. Un dato puede pasar todos los controles de formato y estar mal. La dimensión que más se subestima es la última. Un dato limpio del que nadie puede explicar cómo se limpió sirve menos que un dato sucio documentado.

3.8 La regla del dato crudo intacto

De la trazabilidad sale la regla que ordena todo el taller:

El archivo original nunca se modifica. Se descarga una vez, se guarda aparte y a partir de ahí solo se lee.



Figura 5. El original se conserva; cada resultado sale de una transformación reproducible.

En la práctica son dos carpetas al lado del notebook: *crudo/* para lo que se descargó de la fuente, *generado/* para todo lo que produce el notebook. *generado/* se puede borrar entera y el notebook la reconstruye cuando se ejecuta de arriba hacia abajo.

4. EXPLORAR UN CONJUNTO DE DATOS

4.1 Antes del archivo, la ficha

Un conjunto de datos bien publicado trae dos cosas al lado del archivo. Los **metadatos del conjunto** dicen quién lo publica, cada cuánto se actualiza, qué período cubre y con qué licencia se puede usar. El **diccionario de datos** tiene una fila por columna, con su nombre, su tipo, su unidad y qué significa.

Tres filas del diccionario de la tabla de la sección 3.4 se ven así:

- **id** — entero. Identifica el hogar. Es una etiqueta y no se suma.
- **ingreso** — decimal, en pesos. Ingreso mensual total declarado por el hogar. Vacío si el hogar no contestó.
- **fecha_visita** — fecha, en formato ISO 8601. Día en que el encuestador visitó el hogar.

Con eso ya sabés que **id** no se promedia y que un vacío en **ingreso** es una negativa a contestar y no un error de carga.

El [Perfil Nacional de Metadatos](#) es el estándar que sigue datos.gob.ar. Una planilla que te pasan por correo casi nunca lo trae, y ahí lo primero que hacés es pedirlo: sin diccionario, una columna llamada **t_28** es indescifrable.

4.2 Las cinco preguntas de la primera mirada

Cuando abrís un conjunto de datos por primera vez, hacés siempre las mismas cinco preguntas, en este orden. Es el diagnóstico mínimo antes de tocar nada.

1. **¿Qué tamaño tiene?** Cuántas filas y cuántas columnas. Si el número de columnas no es el que esperabas, algo se leyó mal.
2. **¿Cómo se llaman las columnas?** Los nombres reales, con sus mayúsculas, sus espacios y sus acentos. Un nombre con un espacio de más al final es una fuente clásica de errores.
3. **¿Qué tipo tiene cada columna?** Si es número, texto, fecha o valor lógico. Una columna que debería ser numérica y aparece como texto está avisando que adentro hay algo que no es un número.
4. **¿Cuánto falta?** Cuántos valores vacíos hay en cada columna. No todas las ausencias pesan igual: que falte el 2% de una variable secundaria es distinto de que falte el 40% de la principal.
5. **¿Hay repetidos?** Cuántas filas están duplicadas por completo. Un duplicado exacto casi siempre es un error de carga, y conviene resolverlo antes que cualquier otra cosa.

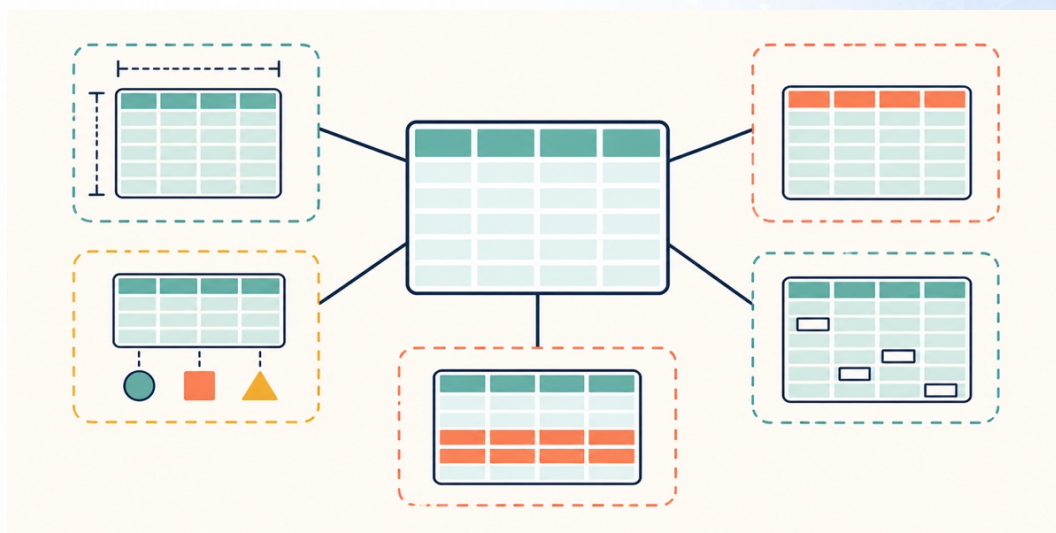


Figura 6. Cinco miradas sobre la misma tabla.

Con la tabla de la sección 3.4 las respuestas son: cuatro filas y seis columnas; los nombres son los del encabezado; id, personas, ambientes e ingreso son números, localidad y fecha_visita son texto; falta un valor en ingreso; no hay filas repetidas.

Dos detalles de esa respuesta valen más que el resto. id se lee como número aunque sea una etiqueta, y sumar identificadores no significa nada. Y ingreso vuelve como decimal, no como entero, justamente porque le falta un valor.

Faltantes. El porcentaje solo no alcanza. Tres preguntas ordenan la decisión.

1. ¿La columna es central para tu análisis o es de contexto? Un 15% de faltantes en una variable secundaria se anota y se sigue. El mismo 15% en la variable principal cambia el trabajo: conseguís el dato o cambiás la pregunta.
2. ¿Los faltantes están repartidos o agrupados? Que falte el ingreso en el 15% de los hogares, mezclados, achica la muestra. Que falten todos los de Concepción es sesgo, y arruina cualquier promedio por localidad.
3. ¿Por qué falta? Que el encuestador no haya preguntado es distinto de que el hogar no haya querido contestar. Lo segundo es informativo: el que no declara su ingreso rara vez es un hogar cualquiera.

Ninguna se contesta mirando la tabla. Se contestan con el diccionario, con la ficha o preguntándole a quien publicó el dato.

Duplicados. Acá importa más el motivo que el porcentaje. Dos filas idénticas sobre cuatrocientas casi siempre son un error de carga y se borran. Antes fijate si tenían que ser idénticas: dos visitas al mismo hogar el mismo día son dos observaciones legítimas, y el archivo no las distingue. Y si se repite el id pero el resto difiere, no borres nada: eso es un conflicto, y hay que decidir qué fila vale.

4.3 La herramienta ya decidió por vos

Cuando un programa lee un CSV, tiene que adivinar el tipo de cada columna, porque el archivo no lo dice. Mira los valores, ve que todos son dígitos y decide que la columna es numérica. Casi siempre acierta, y cuando se equivoca no avisa.

Tres casos frecuentes:

- **La fecha queda como texto.** El programa mira 2025-03-14 y lo que ve es una cadena de caracteres. Mientras siga siendo texto no podés restar dos fechas ni ordenar por mes.
- **Un identificador pierde los ceros de adelante.** 0074 es un código de producto para vos. Para el programa es el número 74, y el cero desaparece. El mismo problema afecta a los números de documento y a los códigos de barras.
- **Los faltantes no siempre están vacíos.** La forma correcta de marcar una ausencia es dejar el lugar vacío, y hay una lista de textos que la herramienta ya reconoce sola, como N/A o NULL. El problema son los otros. Con s/d o -, la columna deja de ser numérica y el conteo de faltantes te da cero cuando en realidad falta el 30%. Con 999, peor todavía: la columna sigue siendo numérica, no falta nada según el conteo, y ese 999 se promedia como si fuera un valor real.

La pregunta 3 es el momento de revisar qué supuso la herramienta y corregirlo antes de seguir.

5. FORMATOS: QUÉ SE CONSERVA Y QUÉ SE PIERDE

5.1 Cinco formatos para la misma tabla

Lo que hay que mirar al convertir de un formato a otro es qué sobrevive.

Formato	Legible	Tipos	Varias tablas	Uso típico
CSV	Sí	No	No	Intercambio universal
XLSX	No	Parcial	Sí	Trabajo manual
JSON	Sí	Parcial	Sí	Interfaces web
Parquet	No	Sí	No	Volumen grande
CSV.GZ	No	No	No	Transporte

Parcial quiere decir cosas distintas en cada fila. XLSX guarda números, texto y fechas, pero no categorías. JSON guarda números y texto, y no tiene tipo fecha: eso vuelve como texto. *Varias tablas* quiere decir que un solo archivo puede contener más de una, como las hojas de un Excel.

Para elegir no hace falta comparar las cinco columnas. Alcanza con preguntar a dónde va el archivo.

- **Se lo mandás a alguien.** CSV. Lo abre cualquiera, sin instalar nada.
- **Lo van a editar a mano.** XLSX. Es el formato de la planilla.
- **Lo consume una página o un sistema.** JSON.
- **Es un paso intermedio tuyo.** Parquet. Conserva los tipos y pesa poco.
- **Hay que moverlo y es grande.** CSV.GZ. Adentro sigue siendo un CSV.

Cuando dudes entre dos, elegí el más simple. Un CSV que cualquiera puede abrir sirve más que un Parquet que nadie sabe leer.

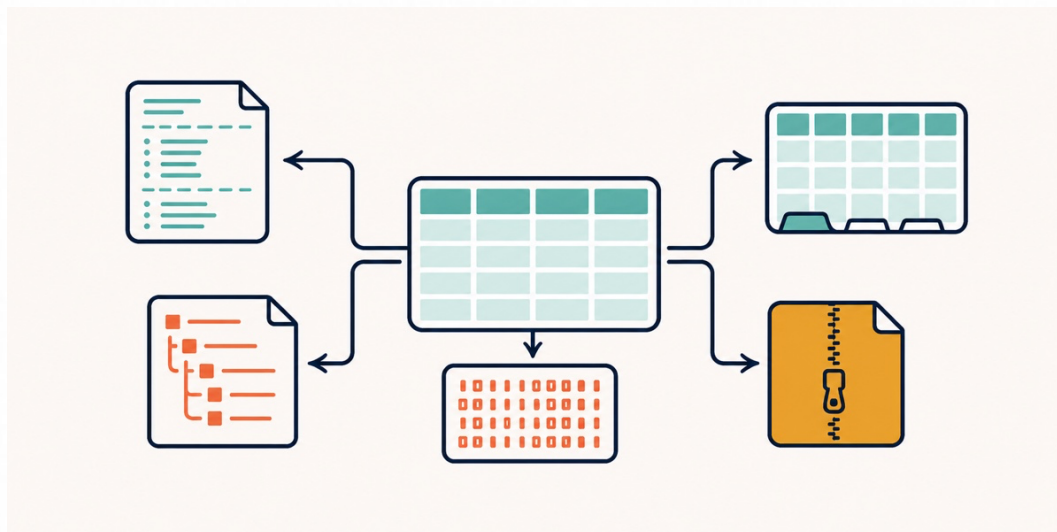


Figura 7. El contenido puede ser el mismo; lo que conserva cada formato, no.

Dos distinciones que suelen mezclarse. Abierto no implica legible: Parquet es un formato abierto y documentado, y es ilegible en un editor de texto. Comprimido no implica propietario: un `.csv.gz` es un CSV común adentro de un envoltorio estándar, y lo abre cualquier herramienta.

5.2 El tipo que se pierde

Un CSV es texto plano: no tiene dónde anotar que una columna es fecha, otra entero y otra categoría.

El recorrido típico es este. Lees un CSV, la columna `fecha_visita` viene como texto, la convertís a fecha, hacés tu trabajo y guardás el resultado en CSV. Al día siguiente abrís ese archivo y `fecha_visita` volvió a ser texto. La conversión se perdió, porque el CSV no tenía dónde guardarla.

Con Parquet no pasa: además de los datos, el archivo guarda una ficha que dice de qué tipo es cada columna. Eso es el **esquema**. La columna vuelve como fecha.

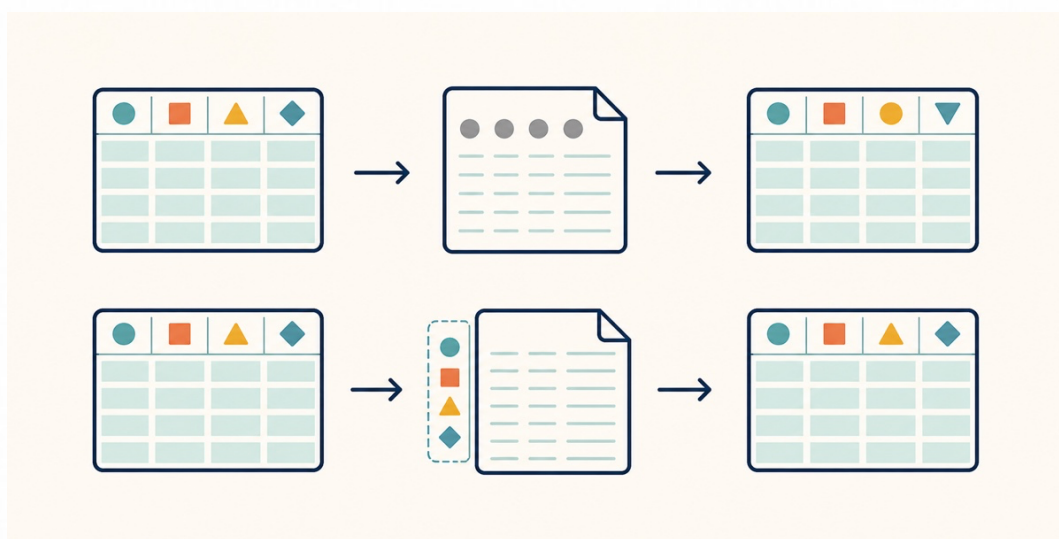


Figura 8. El texto plano no tiene dónde anotar el tipo. El formato con esquema, sí.

Ninguno de los dos está mal. La decisión práctica:

CSV para entregar y para que otro lo abra. Parquet para trabajar y para guardar resultados intermedios.

Si el destino final es CSV, el código que reconstruye los tipos es parte del entregable: mandás el CSV y el notebook que lo lee bien, no el CSV solo.

Perder el tipo de una fecha es más grave de lo que parece. `03/04/2025` es el 3 de abril para nosotros y el 4 de marzo para un lector estadounidense: el mismo texto, dos fechas distintas, y el archivo no dice cuál.

ISO 8601 escribe año, mes y día, de mayor a menor y con ancho fijo: `2025-04-03`. No admite dos lecturas, y el orden alfabético de esas fechas coincide con el cronológico. Es lo que exige la guía de datos abiertos de la Nación: usalo en todo archivo que generes.

5.3 Lo que Excel te cambia sin preguntar

Excel no es un visor: interpreta lo que abre y después guarda su interpretación. Tres pérdidas son silenciosas e irreversibles.

Un identificador como MAR-1 o 1/2 lo convierte en fecha, y el valor original no vuelve. Un número de más de quince dígitos lo redondea, y los dígitos que sobran quedan en cero. Y si el archivo tiene más de 1.048.576 filas, carga las primeras y descarta el resto sin decir cuántas perdió.

Hay dos maneras de evitarlo. La primera es no abrir el CSV con doble clic. Abrió Excel vacío y andá a **Datos** → **Obtener datos** → **Desde texto/CSV**. Excel muestra una vista previa y te deja fijar el tipo de cada columna antes de importar: las que son códigos, ponelas como texto.

La segunda es la del taller: no pasar por Excel. El notebook lee el original y Excel no lo toca nunca. Si igual lo abriste con doble clic, cerralo sin guardar.

Esto no es un problema de principiantes. Un [relevamiento de 2016](#) encontró nombres de genes convertidos en fechas en uno de cada cinco artículos que publican listas de genes en Excel como material suplementario.

6. CODIFICACIÓN DE CARACTERES

6.1 Qué es un *encoding*

Una computadora guarda números. Una letra no es un número. Hace falta una tabla de equivalencias que diga qué número corresponde a cada carácter, y esa tabla se llama **encoding**.

Todo el problema de esta sección sale de una sola frase:

Un archivo de texto no dice en qué encoding está. Son bytes, nada más.

El que lo abre tiene que suponerlo, y hoy casi todas las herramientas suponen UTF-8. Estas son las tablas que vas a encontrar:

Encoding	Cubre	Bytes
ASCII	Inglés	1
Latin-1	Europa occidental	1
CP1252	Latín-1 ampliado	1
UTF-8	Todo Unicode	1 a 4

UTF-8 es el estándar actual y es el que hay que usar siempre. Aun así vas a seguir recibiendo archivos en Latin-1 y en CP1252, porque los generan sistemas viejos y Excel en castellano.

Donde las tablas coinciden, la suposición no importa. Las letras sin acento, los dígitos y las comas tienen el mismo número en todas. Las tablas se separan justo en los caracteres que nos importan en castellano. La ñ de tamaño ocupa dos bytes en UTF-8, `\xc3\xba`, y un solo byte en Latin-1, `\xf1`. La notación `\x` quiere decir que lo que sigue es un byte escrito en hexadecimal, que es la forma habitual de nombrarlos.

UTF-8 usa una cantidad variable de bytes por carácter, así que necesita marcar dónde empieza cada uno. Los bytes de las letras sin acento valen solos. Los demás vienen en grupo: el primero anuncia cuántos bytes tiene el carácter y los que siguen son la continuación. En Latin-1 no hay nada de eso: un byte, un carácter, siempre.

Por eso un archivo con pocos acentos puede pasar años sin dar problemas, hasta que aparece un nombre con ñ.

También vas a ver `utf-8-sig`: es UTF-8 y además saca el BOM, una marca invisible de tres bytes que algunos programas ponen al principio para declarar el encoding. Si el primer nombre de columna aparece como `ï»¿id`, es eso.

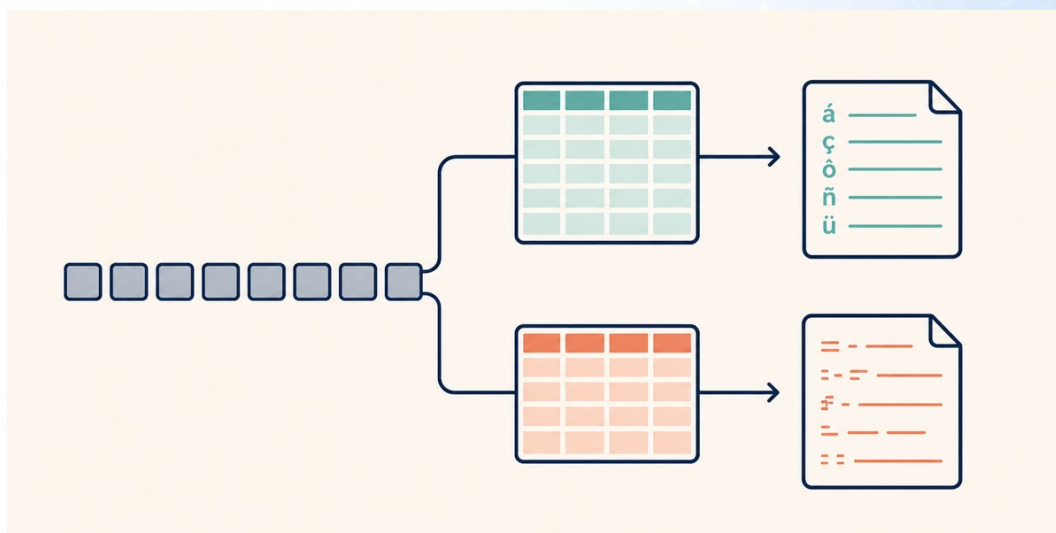


Figura 9. Los bytes no cambian: cambia la tabla con la que se interpretan.

6.2 El error ruidoso

Si lees un archivo Latin-1 suponiendo UTF-8, el programa corta. Tomemos la tabla de la sección 3.4, guardada en Latin-1: el primer carácter con acento es la ó de Córdoba, que en esa tabla es el byte `\xf3`.

En Latin-1 ese byte **es** la ó. En UTF-8 el mismo byte significa otra cosa: anuncia que empieza un carácter de cuatro bytes. Como lo que viene atrás no es la continuación que UTF-8 espera, el decodificador se detiene:

```
'utf-8' codec can't decode byte 0xf3 in position 88: invalid continuation byte
```

El mensaje dice qué encoding supuso, qué byte no supo interpretar y en qué posición estaba.

Para resolverlo, probá los candidatos y mirá cuál produce texto legible: utf-8, latin-1, cp1252 y utf-8-sig, por si lo único que sobra es un BOM. Latin-1 y CP1252 comparten la tabla de la ñ y de las vocales acentuadas, así que las dos van a dar texto legible y el archivo solo no alcanza para decidir.

“Funciona” no quiere decir “es correcto”. Quiere decir “no explotó”.

6.3 El error silencioso

Ahora al revés: un archivo bien hecho en UTF-8, leído como si fuera Latin-1.

Latin-1 tiene un carácter asignado para cada uno de los 256 bytes posibles, así que nunca falla: siempre encuentra algo que mostrar. No hay ningún error, y en la pantalla aparece esto:

tamaÃ±o	en vez de	tamaño
CÃ³rdoba	en vez de	Córdoba
ConcepciÃ³n	en vez de	Concepción

Eso tiene nombre propio: **mojibake**, del japonés “caracteres transformados”. Es la firma inconfundible de UTF-8 leído como Latin-1: la ñ, que en UTF-8 son dos bytes, aparece partida en dos caracteres. Si ves una Ã donde debería haber un acento, ya sabés qué pasó.

El error ruidoso corta en el acto y no rompe nada. El silencioso no avisa, corrompe los datos y aparece cuando alguien mira, semanas después, en un informe. Cuando puedas elegir, preferí que falle fuerte y temprano.



Figura 10. El que corta se paga con una tarde. El que sigue, con el informe.

¿Y si ya guardaste el archivo con el texto roto? Casi siempre se recupera: los bytes están todos, solo que se les aplicó la tabla equivocada, y ese camino se puede desandar. Lo que no vuelve son los caracteres que la herramienta reemplazó por ? o por □. La vía corta, igual, es volver al original y leerlo de nuevo. Para eso existe la regla del dato crudo intacto.

6.4 El CSV argentino

Excel en castellano exporta los CSV con separador de campos ;, separador decimal , y encoding CP1252 o Latin-1. Vas a recibir muchos archivos así:

```
id;localidad;personas;ingreso
1;Ramallo;3;185000,50
2;Córdoba;5;240500,75
```

Los dos primeros cambios van juntos por una razón lógica. Si el decimal es la coma, la coma no puede ser también el separador de campos, porque 3,5 sería ambiguo. Por eso el separador se corre a ;.

Si abris ese archivo sin avisar nada, el programa busca comas y no encuentra ninguna en el encabezado. El resultado es una tabla de **una** columna donde tenía que haber cuatro:

```
columnas: ['id;localidad;personas;ingreso']
filas:    '1;Ramallo;3;185000' -> 50
          '2;Córdoba;5;240500' -> 75
```

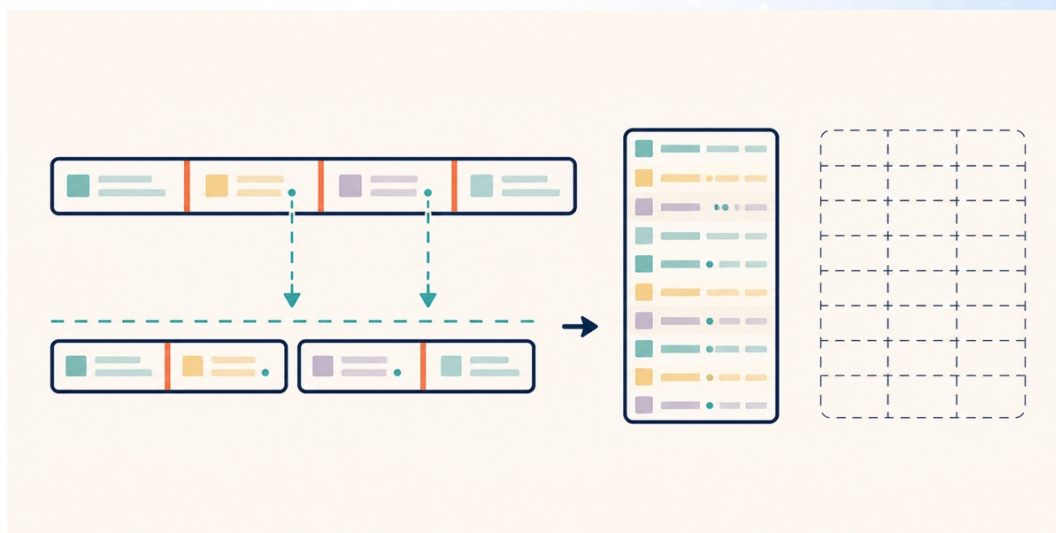



Figura 11. No hay error: la herramienta cortó donde le pediste, y ese lugar estaba mal.

No hubo ningún error. El programa hizo al pie de la letra lo que le pediste, y el resultado es inservible.

El arreglo no es tocar el archivo. Es declarar al abrirlo lo que el archivo no dice: separador ; , decimal , y encoding cp1252. Con esas tres cosas, la tabla vuelve a tener cuatro columnas.

6.5 Lista de verificación

Cuando abras un CSV que no conocés, en este orden:

1. **Mirá el tamaño.** ¿La cantidad de columnas es la que esperabas? Si no, revisá el separador de campos.
2. **Mirá los acentos.** Si aparece una Ñ, leelo con otro encoding.
3. **Mirá los tipos.** ¿Las columnas numéricas son numéricas? Si una que debería serlo quedó como texto, revisá el separador decimal.

Todo lo que decidas ahí, qué encoding supusiste y qué separador usaste, va anotado. Es un supuesto tuyo y no un hecho del archivo.

7. DATOS NO ESTRUCTURADOS

Las imágenes, el sonido y el video también son datos, y valen para ellos las mismas preguntas: qué se conserva, qué se pierde y qué dice el archivo además del contenido.

Para analizarlos junto con el resto, primero hay que extraerles una variable medible: de una foto, la fecha en que se sacó o la cantidad de píxeles oscuros; de una grabación, la duración y el volumen medio. Esa variable sí entra en una columna.

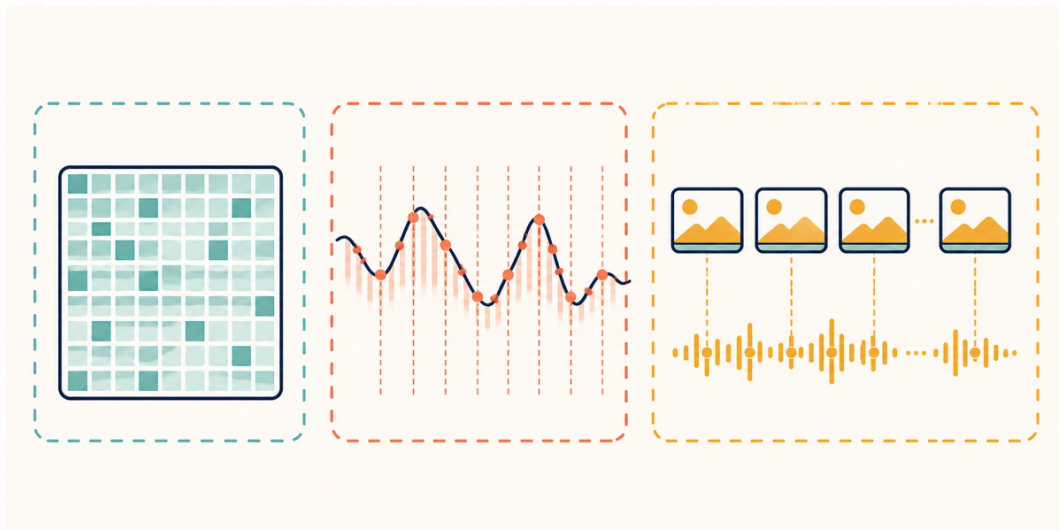


Figura 12. Imagen, sonido y video guardan mediciones discretas de una realidad continua.

7.1 Una imagen es una grilla de números

Una imagen digital es una grilla de puntos, los **píxeles**. Cada píxel guarda tres números, uno por canal de color: rojo, verde y azul. Cada número va de 0 a 255, así que ocupa un byte.

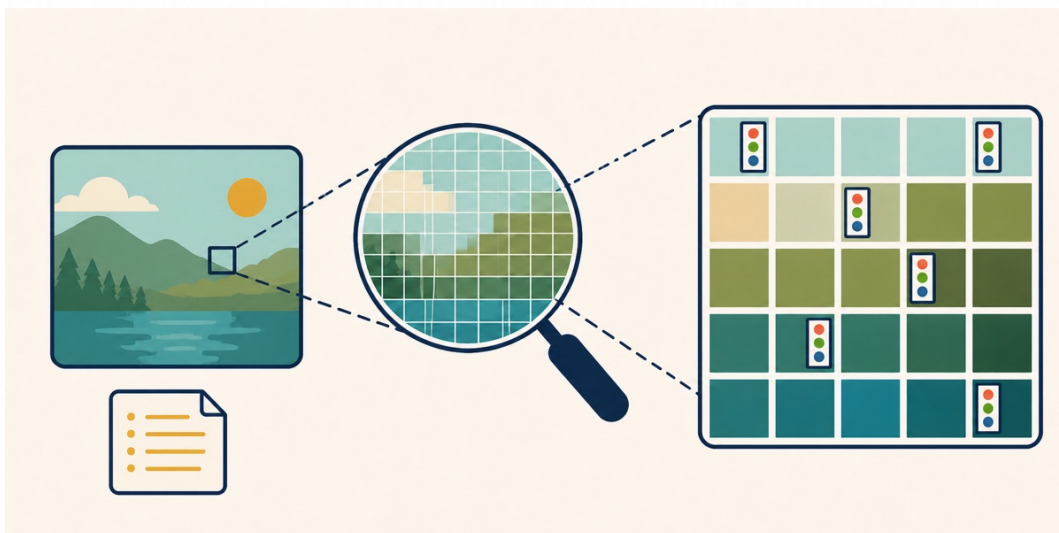


Figura 13. Cada punto guarda tres mediciones de color, una por canal.

De ahí sale el peso de una imagen sin comprimir, a 8 bits por canal, que es una multiplicación:

$$4000 \times 3000 \text{ píxeles} \times 3 \text{ canales} = 36.000.000 \text{ bytes} \approx 36 \text{ MB}$$

En el celular esa foto pesa 3 MB: la diferencia es la compresión.

La **resolución** es la cantidad de píxeles, y determina cuánto detalle hay. Una imagen de 4000 × 3000 se puede achicar a 800 × 600 y sigue viéndose bien en pantalla. Al revés no funciona: agrandar una imagen chica no inventa el detalle que no estaba.

7.2 El sonido es una serie de muestras

Un sonido es una onda continua, y una computadora no puede guardar algo continuo. Lo que hace es medir la altura de la onda muchas veces por segundo. Cada medición es una **muestra**.

La **frecuencia de muestreo** dice cuántas veces por segundo se mide: 44.100 muestras por segundo es el estándar del CD, y 48.000 el de video. La **profundidad de bits** dice con cuánta precisión se guarda cada medición, y 16 bits, o sea 2 bytes, es lo habitual.

Un minuto de audio en calidad de CD, en estéreo y sin comprimir:

$$44.100 \times 2 \text{ bytes} \times 2 \text{ canales} \times 60 \text{ segundos} \approx 10,6 \text{ MB por minuto}$$

Es la misma idea que la resolución: más muestras, más detalle y más peso.

7.3 El video multiplica

Un video es una secuencia de imágenes, más una pista de audio. A 25 cuadros por segundo y resolución 1920 × 1080, un solo segundo sin comprimir ocupa unos 155 MB, y un minuto pasa de los 9 GB. Por eso el video siempre está comprimido, y con métodos que aprovechan que un cuadro se parece mucho al anterior para guardar solo las diferencias.

Estas cuentas se hacen antes de pedir el dato, no después. Son las que te dicen si “mandame las fotos de las 500 sucursales” son 2 GB o 200, y si eso entra en el disco, en el correo y en el presupuesto.

7.4 Compresión con y sin pérdida

Hay dos familias, y la diferencia importa:

- **Sin pérdida.** El archivo comprimido contiene toda la información del original. Al descomprimirlo recuperarás exactamente los mismos bytes. PNG para imágenes, FLAC para sonido, ZIP y GZIP para cualquier cosa.
- **Con pérdida.** El método descarta información que el ojo o el oído casi no perciben, y por eso logra archivos mucho más chicos. JPEG para imágenes, MP3 para sonido, la mayoría de los formatos de video. Lo descartado no vuelve.

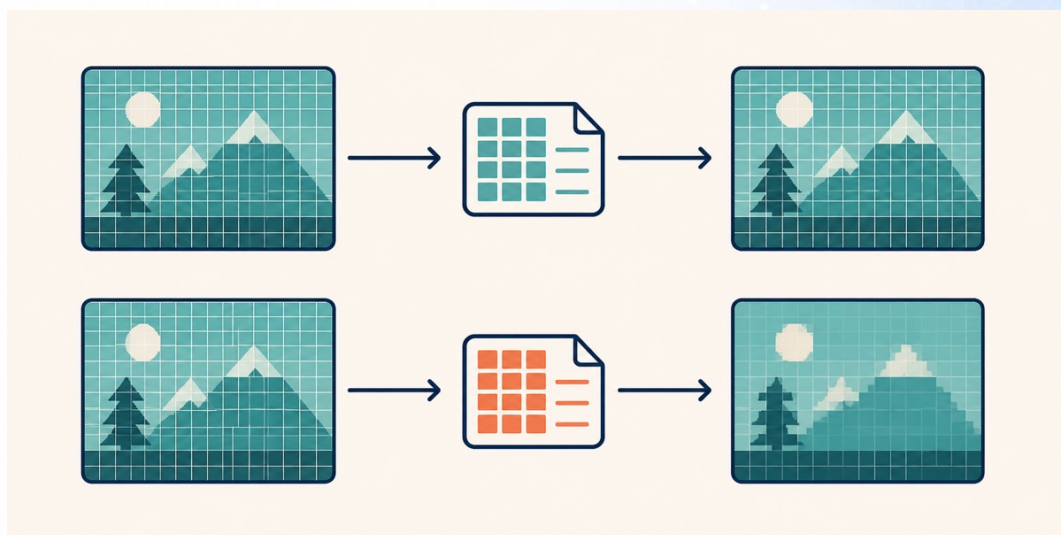


Figura 14. Un mismo original, dos caminos de vuelta.

La pérdida se acumula. Cada vez que abrís un JPEG y lo volvéis a guardar, el método descarta un poco más, y después de varias vueltas la degradación se nota. Con un PNG podés hacer eso mil veces sin perder nada.

La regla es la misma que con el dato crudo: guardá el original sin pérdida y generá las versiones comprimidas a partir de él, no al revés.

7.5 Contenido y metadatos

Un archivo de imagen trae la imagen, y además trae datos sobre la imagen: cuándo se sacó la foto, con qué cámara, con qué apertura y, si el celular tenía el GPS encendido, en qué coordenadas. Ese segundo conjunto se llama **metadatos**, y en las fotos usa el estándar EXIF.

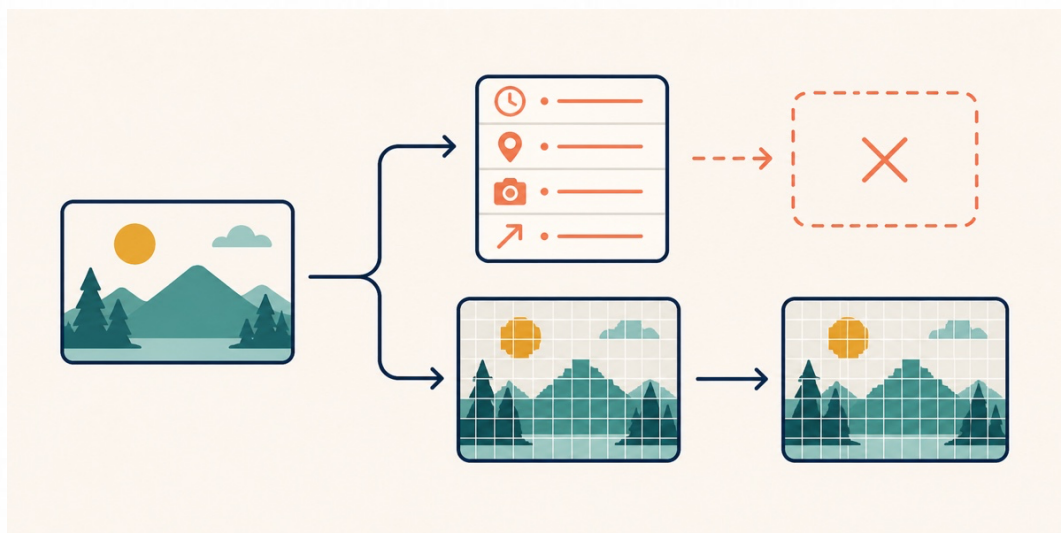


Figura 15. Los píxeles muestran la imagen; los metadatos cuentan cómo y dónde nació.

Dos razones por las que esto importa acá.

La primera es de privacidad. Si publicás fotos sacadas con el celular, estás publicando también las coordenadas donde las sacaste. Las redes y la mensajería suelen borrar los metadatos al recomprimir la foto, pero un archivo que mandás por correo, o por mensajería como documento adjunto en vez de como foto, los conserva enteros.

La segunda es de reproducibilidad. Los metadatos son a veces el único registro de dónde salió un archivo. Un caso concreto: 500 fotos de una inspección de sucursales. La fecha y las coordenadas EXIF son lo único que dice cuándo y dónde se tomó cada una. Si el paso de conversión no las copia, quedan 500 imágenes sin fecha ni lugar, y nadie va a poder decir con qué instrumento se tomó el dato.

8. LOS EJEMPLOS EN GOOGLE COLAB

Casi todo lo que describe este apunte está también en un notebook, con el código ya escrito y listo para ejecutar: [01 tipos y formatos.ipynb](#)

Una celda del notebook falla a propósito: es la del error de encoding, y está avisada dos veces antes de llegar. Cuando aparezca el bloque rojo, seguí con la celda siguiente.